โครงงาน

เกมเขาวงกตตัวเลข

(Number Maze)

จัดทำโดย

6804062611033 นายวัชรเศรษฐ์ ว่องรัตน์วานิช

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ชื่อ Project

หัวข้อ	รายละเอียด
ชื่อ Project	เกมเขาวงกตตัวเลข (Number Maze)
ประเภท	เกม (Java Desktop Application)
ผู้จัดทำ	6804062611033 นายวัชรเศรษฐ์ ว่องรัตน์วานิช
อาจารย์ผู้สอน	ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์
รายวิชา	040613204 Object-Oriented Programming
Source Code	https://github.com/Watcharaset/NumberMaze
ไฟล์ jar สำหรับเล่น	https://github.com/Watcharaset/NumberMaze/releases

สารบัญ

ชื่อ Project 1

บทที่ 1 บทนำ 3

1.1 ที่มาและความสำคัญของโปรเจ็ค 3

1.2 ประเภทของโครงการ 3

1.3 ประโยชน์ 3

1.4 ขอบเขตของโครงการ 3

บทที่ 2 ส่วนการพัฒนา 8

2.1 เนื้อเรื่องย่อและวิธีการเล่น 8

2.2 คลาสไดอะแกรม พร้อมคำอธิบาย 9

2.3 รูปแบบการพัฒนา Application / Applet 15

2.4 อธิบายส่วนของโปรแกรมตามแนวคิดเชิงวัตถุ 17

2.5 หน้าจอ GUI 20

2.6 Event handling ในหน้าจอ 25

2.7 อัลกอริทึมที่สำคัญในโปรแกรม 28

บทที่ 3 สรุป 31

3.1 ปัญหาที่พบระหว่างการพัฒนา 31

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร 31

3.3 คำแนะนำสำหรับผู้สอน 32

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของโปรเจ็ค

โครงงานนี้จัดทำขึ้นเพื่อวัดผลความสามารถในการเรียนวิชา Object-Oriented Programming โดยนำความรู้เรื่องการเขียนโปรแกรมเชิงวัตถุด้วยภาษา Java มาสร้างเป็นเกมบนคอมพิวเตอร์ ผู้จัดทำเลือกสร้างเกมแนวเขาวงกตผสมโจทย์คณิตศาสตร์ เนื่องจากเกมประกอบด้วยวัตถุหลายชนิดที่มีลักษณะร่วมกันแต่มีพฤติกรรมต่างกัน เช่น ประตู ไอเทม และอุปสรรค จึงเหมาะกับการนำหลักการ Inheritance, Polymorphism และ Abstraction มาใช้ได้อย่างเป็นรูปธรรม

นอกจากนี้ เกมที่ต้องคิดเลขยังช่วยให้ผู้เล่นได้ฝึกทักษะคณิตศาสตร์ไปพร้อมกับความสนุก โดยเฉพาะเรื่องลำดับความสำคัญของเครื่องหมาย (คูณ/หาร ก่อน บวก/ลบ) ซึ่งเป็นเรื่องที่มักคำนวณผิดเมื่อคิดเลขในใจ

1.2 ประเภทของโครงการ

- เกมบนคอมพิวเตอร์ (Desktop Game Application) แนวเขาวงกตไขปริศนา (Puzzle Maze)
- พัฒนาด้วยภาษา Java และ Java Swing ทำงานเป็น Java Application บนระบบปฏิบัติการใดก็ได้ที่ติดตั้ง Java 17 ขึ้นไป

1.3 ประโยชน์

1. ช่วยพัฒนาทักษะการคิดเลขในใจ การลำดับความสำคัญทางคณิตศาสตร์ และการวางแผนแก้ปัญหาอย่างเป็นระบบ
2. ผู้จัดทำได้นำแนวคิดการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming) มาประยุกต์ใช้กับงานจริง
3. ผู้จัดทำได้ฝึกการออกแบบโปรแกรมด้วย UML Class Diagram การเขียน GUI และ Event handling ด้วย Java Swing
4. ผู้จัดทำได้ฝึกใช้ Git และ GitHub ในการจัดการเวอร์ชันของโค้ดและเผยแพร่ผลงาน

1.4 ขอบเขตของโครงการ

1) ความต้องการของระบบ (Functional Requirements)

- ผู้เล่นควบคุมนักสำรวจด้วยปุ่ม W A S D หรือปุ่มลูกศร และเดินผ่านกำแพงไม่ได้
- ผู้เล่นเก็บตัวเลข (0–9) และเครื่องหมาย (+ − × ÷) ที่กระจายอยู่ในด่านเข้ากระเป๋า (Inventory) ได้

- เมื่อเดินชนประตูภารกิจ
  ระบบแสดงหน้าต่างให้เลือกไอเทมมาจัดเรียงเป็นสมการให้ได้ผลลัพธ์ตรงตามเป้าหมาย
- ระบบคำนวณตามลำดับความสำคัญทางคณิตศาสตร์จริง และตรวจรูปแบบสมการ เช่น ตัวเลขติดกัน หรือการหารด้วยศูนย์
- ไอเทมที่ใช้ในสมการที่ถูกต้องจะถูกใช้หมดไป ผู้เล่นจึงต้องวางแผนการใช้ไอเทม
- มีอุปสรรค 2 ชนิด คือ กับดักหนามที่โผล่–หดเป็นจังหวะ และผู้พิทักษ์ที่เดินลาดตระเวน หากโดนจะถูกส่งกลับจุดเริ่มต้นของด่าน
- มี 3 ด่านที่แผนที่ อุปสรรค และความยากต่างกัน และกดปุ่ม R เพื่อเริ่มด่านใหม่ได้
- เมื่อผ่านประตูชัยของด่านสุดท้าย ระบบแสดงข้อความ "YOU SURVIVED!"

2) ข้อจำกัดของระบบ

- เล่นได้ 1 คน ด้วยคีย์บอร์ด และหน้าต่างแก้สมการใช้เมาส์คลิก
- ตัวเลขในสมการเป็นเลขหลักเดียว (0–9) และยังไม่รองรับวงเล็บ
- ไม่มีระบบบันทึกความคืบหน้า เมื่อปิดเกมต้องเริ่มด่านแรกใหม่

3) ตารางเวลาการดำเนินโครงการ (ตาม Proposal)

ตารางที่ 1.1 ตารางแผนงาน

ลำดับ	รายการ	13–19 ก.ย.	20–26 ก.ย.	27 ก.ย.–3 ต.ค.	4–10 ต.ค.
1	หารูปตัวละครและทำกราฟิกต่าง ๆ				
2	ศึกษาเอกสารและข้อมูลที่เกี่ยวข้อง				
3	ลงมือเขียนโปรแกรม				
4	จัดทำเอกสาร				
5	ตรวจสอบและแก้ไขข้อผิดพลาด				

4) เอกสาร Proposal ที่เสนอ

รายละเอียดของเกมตามที่เสนอไว้ใน Proposal มีดังนี้

เขาวงกตตัวเลข

รายละเอียดเกม

นักสำรวจที่ติดอยู่ในเขาวงกตลึกลับ ซึ่งเส้นทางแต่ละสายจะถูกปิดกั้นด้วย "ประตูรหัสผ่าน" วิธีเดียวที่จะผ่านไปได้คือการแก้สมการคณิตศาสตร์ให้ถูกต้อง

ผู้เล่นจะต้องเดินสำรวจเพื่อเก็บรวบรวมตัวเลขและเครื่องหมายทางคณิตศาสตร์ที่กระจายอยู่ในด่าน นำมาจัดเรียงเพื่อปลดล็อกเส้นทางสู่ทางออก

วิธีการเล่น

ใช้ WASD ในการเคลื่อนที่ และเมื่อเดินไปชนประตูที่ล็อก หน้าต่าง UI จะแสดงขึ้นมา ผู้เล่นต้องนำตัวเลขและเครื่องหมายที่มีมาจัดเรียงให้ได้ผลลัพธ์ตรงตามเป้าหมาย โดยระบบจะคำนวณตามหลักคณิตศาสตร์จริง

Storyboard



นักสำรวจ



ประตูภารกิจ



ประตูชัย

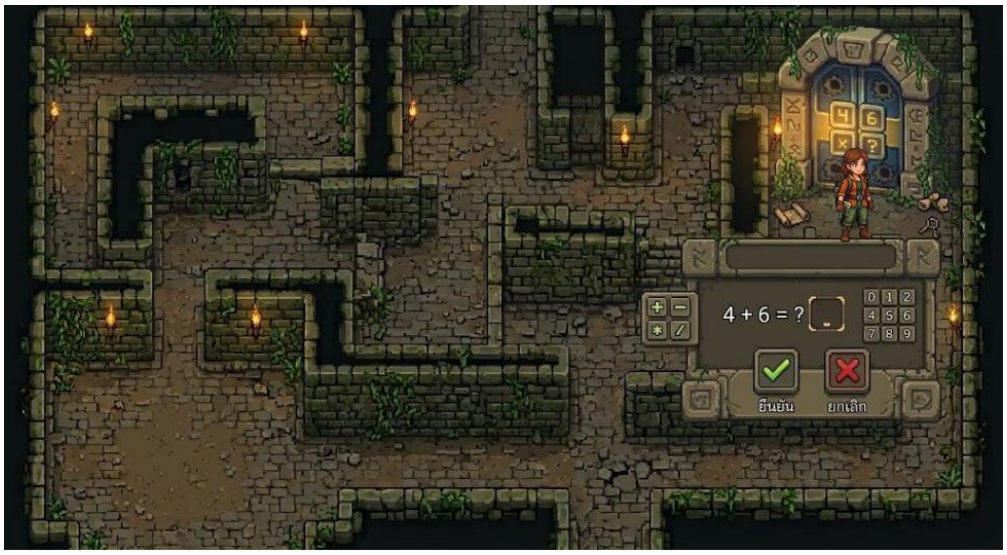
รูปที่ 1.1 ตัวละครในเกม

1) เริ่มเกม



รูปที่ 1.2 Storyboard: เริ่มเกม

2) เมื่อเดินไปที่ประตูจะขึ้นภารกิจขึ้นมา



รูปที่ 1.3 Storyboard: หน้าต่างภารกิจเมื่อเดินชนประตู

3) เมื่อประตูเปิดแล้วเดินเข้าไปจนถึงห้องสุดท้ายจะพบประตูภารกิจและประตูชัย



รูปที่ 1.4 Storyboard: ห้องสุดท้าย

4) เมื่อเคลียร์ประตูภารกิจรองสุดท้ายแล้วเข้าไปในประตูชัยจะขึ้นข้อความ YOU SURVIVED! ขึ้นมา



รูปที่ 1.5 Storyboard: ผ่านประตูชัย

ประโยชน์

ช่วยพัฒนาทักษะการคิดเลขในใจ การลำดับความสำคัญทางคณิตศาสตร์ และการวางแผนแก้ปัญหาอย่างเป็นระบบ

ตารางที่ 1.2 ตารางแผนงาน (ตาม Proposal)

ลำดับ	รายการ	13–19 ก.ย.	20–26 ก.ย.	27 ก.ย.–3 ต.ค.	4–10 ต.ค.
1	หารูปตัวละครและทำกราฟิกต่าง ๆ				
2	ศึกษาเอกสารและข้อมูลที่เกี่ยวข้อง				
3	ลงมือเขียนโปรแกรม				
4	จัดทำเอกสาร				
5	ตรวจสอบและแก้ไขข้อผิดพลาด				

บทที่ 2 ส่วนการพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

เนื้อเรื่องย่อ

นักสำรวจคนหนึ่งติดอยู่ในเขาวงกตโบราณที่ลึกลับ เส้นทางแต่ละสายถูกปิดกั้นด้วย "ประตูรหัสผ่าน" ซึ่งจะเปิดออกก็ต่อเมื่อมีผู้แก้สมการคณิตศาสตร์ที่สลักไว้บนประตูได้ถูกต้อง นักสำรวจต้องเดินสำรวจเขาวงกตเพื่อเก็บตัวเลขและเครื่องหมายที่กระจายอยู่
หลบกับดักหนามและผู้พิทักษ์หินที่เดินลาดตระเวน
แล้วนำสิ่งที่เก็บได้มาเรียงเป็นสมการเพื่อปลดล็อกประตูทีละบาน จนถึงประตูชัยของห้องสุดท้าย และรอดออกมาได้สำเร็จ

ปุ่มควบคุม

ตารางที่ 2.1 ปุ่มควบคุมเกม

ปุ่ม	การทำงาน
W A S D หรือ ปุ่มลูกศร	เดินขึ้น / ซ้าย / ลง / ขวา
R	เริ่มด่านปัจจุบันใหม่ (ไอเทมและประตูกลับสู่สถานะเริ่มต้น)
Enter	เล่นใหม่ตั้งแต่ด่านแรก หลังจบเกม
เมาส์คลิก	เลือกไอเทมและกดปุ่มในหน้าต่างแก้สมการ

องค์ประกอบในเกม

ตารางที่ 2.2 สัญลักษณ์และองค์ประกอบในเขาวงกต

ภาพ	ชื่อ	คำอธิบาย
	นักสำรวจ	ตัวละครที่ผู้เล่นควบคุม
	ไอเทมตัวเลข	ตัวเลข 0–9 (สีเหลือง) เดินทับเพื่อเก็บเข้ากระเป๋า
	ไอเทมเครื่องหมาย	เครื่องหมาย + − * / (สีฟ้า) เดินทับเพื่อเก็บเข้ากระเป๋า
	ประตูภารกิจ	ป้ายด้านล่างบอกเป้าหมาย เช่น =7 ต้องแก้สมการให้ได้ค่านี้จึงจะเปิด
	ประตูชัย	ทางออกของด่าน เดินชนแล้วผ่านด่านทันที
	กับดักหนาม	โผล่ 1 วินาที หด 1.2 วินาที สลับกัน ต้องเดินผ่านตอนหนามหด

ภาพ	ชื่อ	คำอธิบาย
	ผู้พิทักษ์	เดินลาดตระเวนซ้าย–ขวา ห้ามเดินชน
	กำแพง	เดินผ่านไม่ได้

วิธีการเล่น

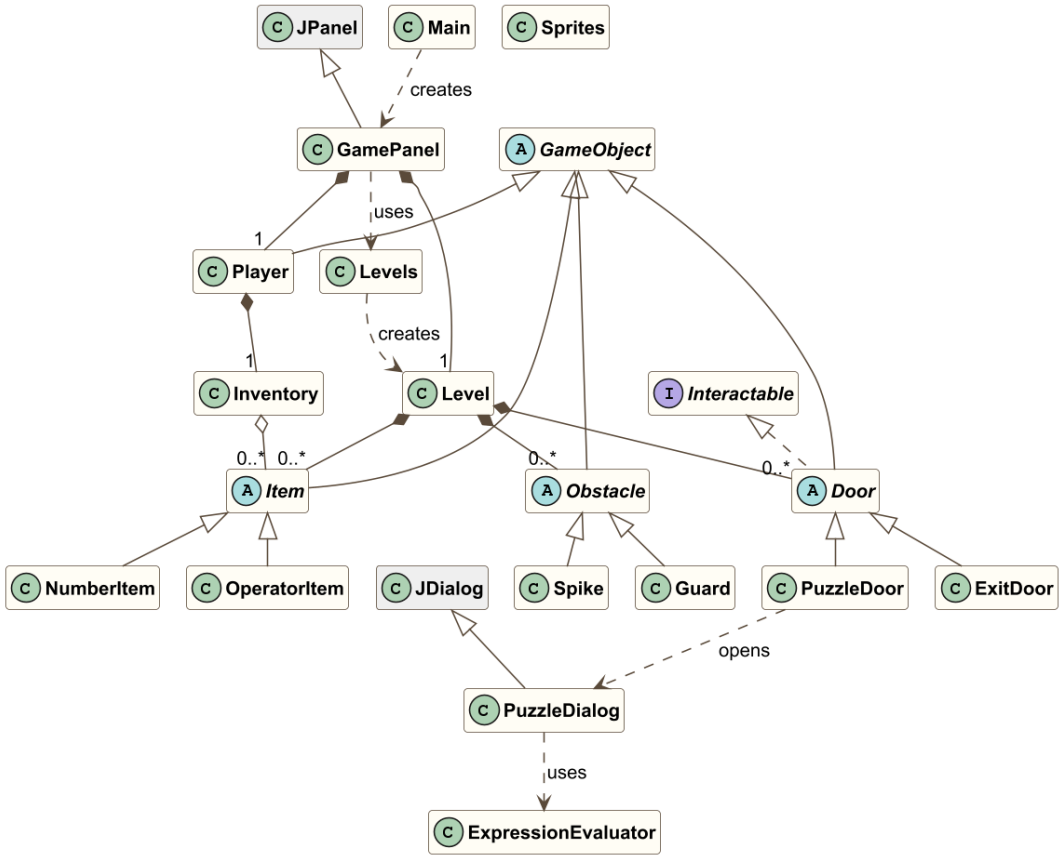
1. เดินสำรวจเขาวงกตด้วยปุ่ม W A S D เพื่อเก็บตัวเลขและเครื่องหมาย
   ไอเทมที่เก็บได้จะแสดงในแถบ "กระเป๋า" ด้านล่างจอ
2. เมื่อเดินชนประตูภารกิจ จะมีหน้าต่างให้คลิกไอเทมในกระเป๋าเรียงเป็นสมการ เช่น 7 * 3 - 5
   แล้วกด "ตกลง"
3. ระบบคำนวณตามหลักคณิตศาสตร์จริง คือคูณ/หารก่อนบวก/ลบ
   ถ้าได้ผลลัพธ์ตรงกับเป้าหมาย ประตูจะเปิด และไอเทมที่ใช้จะหายไปจากกระเป๋า ถ้าไม่ตรง
   ไอเทมยังอยู่ครบและลองใหม่ได้
4. ระวังกับดักหนามและผู้พิทักษ์ ถ้าโดนจะถูกส่งกลับจุดเริ่มต้นของด่าน (ไอเทมในกระเป๋ายังอยู่)
5. วางแผนการใช้ไอเทมให้ดี เพราะไอเทมบางชิ้นต้องเก็บไว้ใช้กับประตูห้องถัดไป
   ถ้าใช้ผิดจนไปต่อไม่ได้ ให้กด R เพื่อเริ่มด่านใหม่
6. เข้าประตูชัยเพื่อไปด่านถัดไป เมื่อผ่านประตูชัยของด่านที่ 3 จะได้รับข้อความ "YOU
   SURVIVED!"

ตารางที่ 2.3 รายละเอียดแต่ละด่าน

ด่าน	ขนาดแผนที่	อุปสรรค	ประตูและเป้าหมาย
1 ห้องฝึกหัด	15 × 11	ไม่มี	2 บาน: =7, =10
2 ทางหนาม	15 × 11	กับดักหนาม	2 บาน: =6, =4
3 ผู้พิทักษ์	21 × 13	หนาม + ผู้พิทักษ์ 2 ตัว + ไอเทมหลอก	3 บาน: =16, =13, =18

2.2 คลาสไดอะแกรม พร้อมคำอธิบาย

โปรแกรมประกอบด้วยคลาสทั้งหมด 20 คลาส (รวม interface 1 ตัว) แบ่งออกเป็น 3 กลุ่ม คือ กลุ่มวัตถุในเกม กลุ่มระบบเกม และกลุ่มเครื่องมือ
ความสัมพันธ์ระหว่างคลาสทั้งหมดแสดงในรูปที่ 2.1 และรายละเอียด Attribute กับ Method ของแต่ละกลุ่มแสดงในรูปที่ 2.2 ถึง 2.4



รูปที่ 2.1 Class Diagram ภาพรวม แสดงคลาสทั้งหมดและความสัมพันธ์ระหว่างคลาส

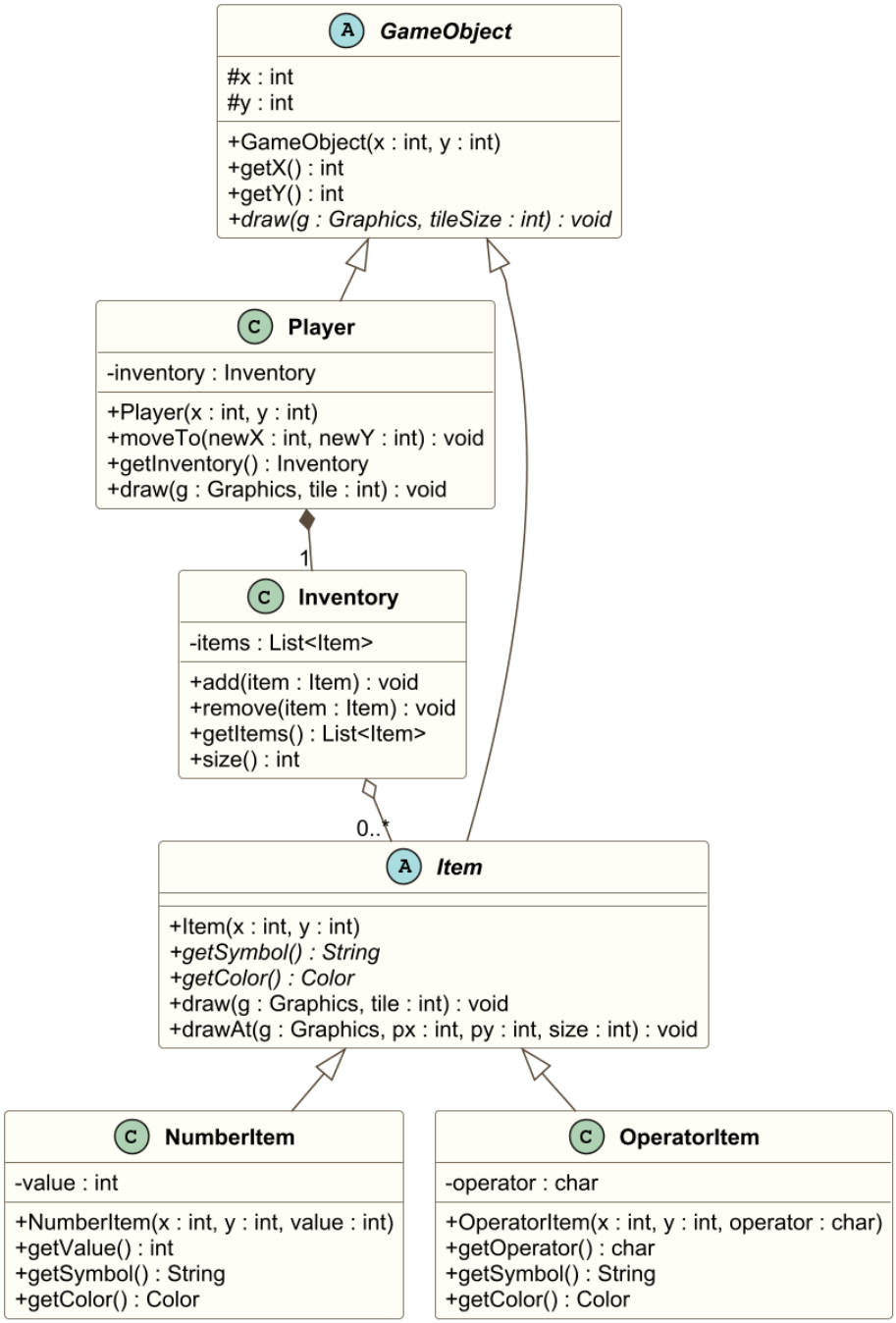
ตารางที่ 2.4 ความหมายของสัญลักษณ์ใน Class Diagram

สัญลักษณ์	ความหมาย
+ / - / #	public / private / protected
ตัวเอียง	abstract class หรือ abstract method
ขีดเส้นใต้	static attribute หรือ static method
ไอคอน C / A / I	Class / Abstract class / Interface
เส้นทึบ หัวสามเหลี่ยมโปร่ง	Inheritance (สืบทอดคลาส)
เส้นประ หัวสามเหลี่ยมโปร่ง	Realization (implements interface)
เส้นทึบ หัวเพชรทึบ	Composition (เป็นเจ้าของ เช่น Player มี Inventory)
เส้นทึบ หัวเพชรโปร่ง	Aggregation (เก็บไว้ เช่น Inventory เก็บ Item)
เส้นประ หัวลูกศร	Dependency (เรียกใช้หรือสร้าง Object ของคลาสนั้น)

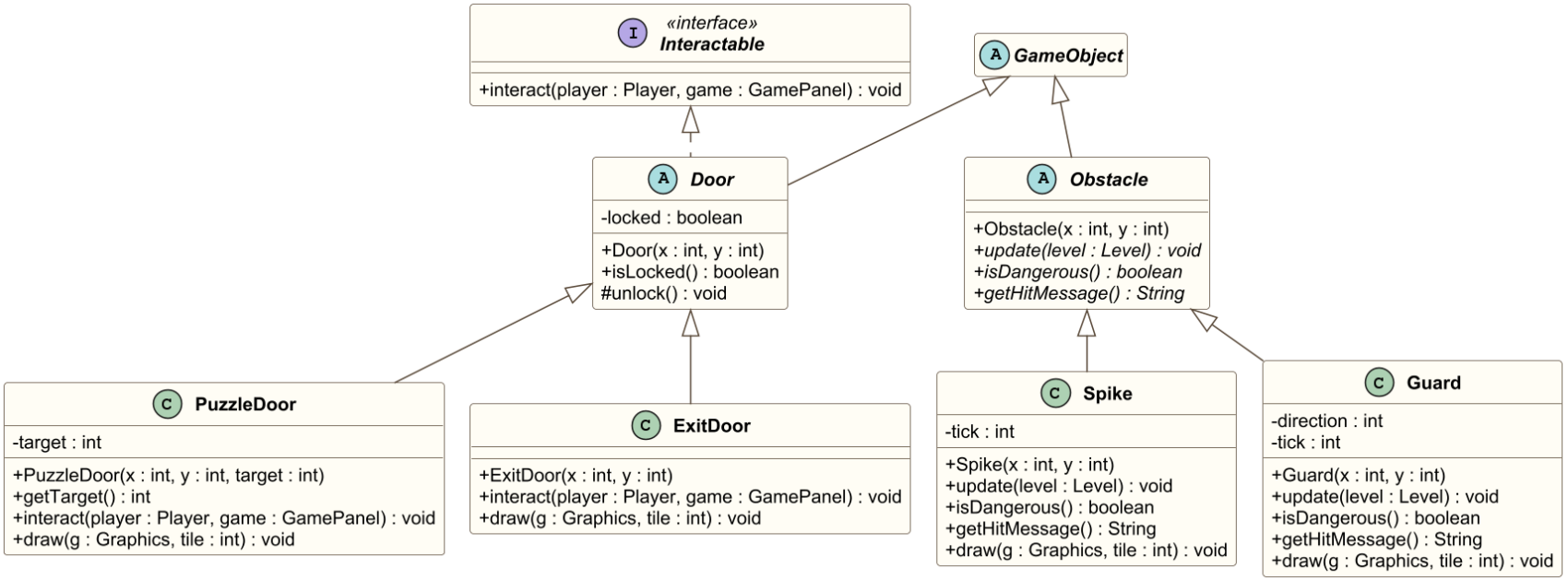
กลุ่มที่ 1: วัตถุในเกม (สืบทอดจาก GameObject)

- **GameObject** เป็น Abstract class ที่เก็บตำแหน่ง x, y ของวัตถุบนตาราง (protected) มีเมธอด getX(), getY() และ Abstract method draw() ที่คลาสลูกทุกตัวต้อง implement เพื่อวาดตัวเอง
- **Player** นักสำรวจที่ผู้เล่นควบคุม มี Inventory เป็นของตัวเอง มีเมธอด moveTo() และ getInventory()
- **Inventory** กระเป๋าของผู้เล่น เก็บไอเทมไว้ใน List ที่เป็น private มีเมธอด add(), remove(), getItems() และ size()
- **Item** Abstract class ของไอเทม มี Abstract method getSymbol() และ getColor() และมีเมธอด drawAt() ที่ใช้วาดไอเทมทั้งบนแผนที่และในแถบกระเป๋า แบ่งเป็น NumberItem (เก็บค่าตัวเลข value) และ OperatorItem (เก็บเครื่องหมาย operator)
- **Interactable** Interface ของสิ่งที่ผู้เล่นเดินชนแล้วมีปฏิกิริยา มีเมธอด interact()
- **Door** Abstract class ของประตู implement Interactable เก็บสถานะ locked โดยมีเพียงคลาสลูกที่เรียก unlock() ได้ แบ่งเป็น PuzzleDoor (ประตูภารกิจ เก็บค่าเป้าหมาย target และเปิดหน้าต่าง PuzzleDialog เมื่อถูกชน) และ ExitDoor (ประตูชัย ชนแล้วผ่านด่าน)
- **Obstacle** Abstract class ของอุปสรรค มี Abstract method update(), isDangerous() และ getHitMessage() แบ่งเป็น Spike (หนามที่โผล่–หดตามจังหวะเวลา) และ Guard (ผู้พิทักษ์ที่เดินซ้าย–ขวา ชนกำแพงแล้วหันกลับ)

รายละเอียดของกลุ่ม Item แสดงในรูปที่ 2.2 และกลุ่มประตูกับอุปสรรคแสดงในรูปที่ 2.3



รูปที่ 2.2 Class Diagram รายละเอียด: GameObject, Player, Inventory และกลุ่ม Item



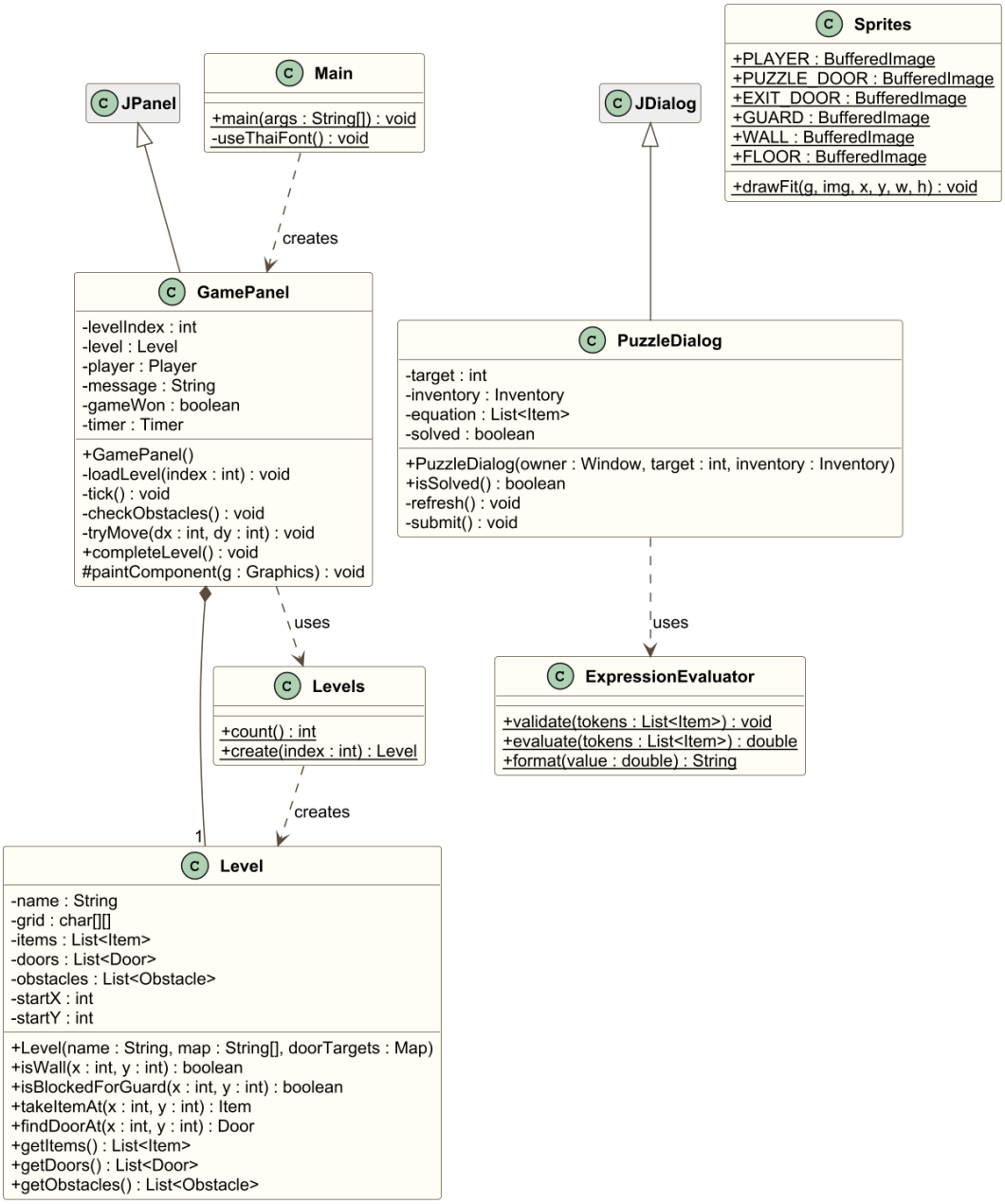
รูปที่ 2.3 Class Diagram รายละเอียด: กลุ่มประตู (Door) และกลุ่มอุปสรรค (Obstacle)

กลุ่มที่ 2: ระบบเกม

- **GamePanel** สืบทอดจาก JPanel เป็นศูนย์กลางของเกม รับปุ่มกดจากคีย์บอร์ด ใช้ Timer ทุก 0.2 วินาทีให้อุปสรรคเคลื่อนที่ วาดหน้าจอทั้งหมด และควบคุมการเปลี่ยนด่านผ่าน loadLevel() และ completeLevel()
- **Level** อ่านแผนที่ที่เขียนเป็นข้อความแล้วสร้าง Object ของไอเทม ประตู และอุปสรรคทั้งหมดในด่าน มีเมธอด isWall(), takeItemAt() และ findDoorAt()
- **Levels** เก็บข้อมูลแผนที่ของทั้ง 3 ด่าน เมธอด create() สร้าง Level ใหม่ทุกครั้ง จึงใช้ในการเริ่มด่านใหม่ได้ด้วย
- **PuzzleDialog** สืบทอดจาก JDialog เป็นหน้าต่างให้ผู้เล่นคลิกเลือกไอเทมมาเรียงเป็นสมการ และตรวจคำตอบ

กลุ่มที่ 3: เครื่องมือ

- **ExpressionEvaluator** ตรวจรูปแบบและคำนวณสมการตามลำดับความสำคัญทางคณิตศาสตร์ (อธิบายในหัวข้อ 2.7)
- **Sprites** โหลดรูปภาพทั้งหมดของเกมเพียงครั้งเดียวตอนเริ่มโปรแกรม
- **Main** จุดเริ่มต้นของโปรแกรม สร้างหน้าต่าง JFrame และตั้งค่าฟอนต์ภาษาไทย



รูปที่ 2.4 Class Diagram รายละเอียด: กลุ่มระบบเกมและกลุ่มเครื่องมือ

2.3 รูปแบบการพัฒนา Application / Applet

โปรแกรมนี้พัฒนาในรูปแบบ **Java Application** (Desktop Application) ไม่ใช่ Applet โดยโปรแกรมเริ่มทำงานที่เมธอด main() ของคลาส Main ซึ่งสร้างหน้าต่าง JFrame แล้วใส่ GamePanel ลงไป ผู้ใช้เปิดเล่นได้โดยดับเบิลคลิกไฟล์ NumberMaze.jar หรือรันคำสั่ง java -jar NumberMaze.jar

```
public class Main {
    public static void main(String[] args) {
        useThaiFont();
        JFrame frame = new JFrame("Number Maze");
        frame.add(new GamePanel());
        frame.pack();
        frame.setResizable(false);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

เหตุผลที่เลือกแบบ Application แทน Applet คือ Applet ต้องทำงานผ่าน Java Plug-in ของเว็บเบราว์เซอร์ ซึ่งเบราว์เซอร์ปัจจุบันเลิกรองรับแล้ว และ Java API ของ Applet ถูกประกาศเลิกใช้ (deprecated) ตั้งแต่ Java 9 การพัฒนาเป็น Application จึงเปิดเล่นได้จริงบนเครื่องคอมพิวเตอร์ทั่วไปที่ติดตั้ง Java

ตารางที่ 2.5 เครื่องมือที่ใช้ในการพัฒนา

หมวด	เครื่องมือ
ภาษา	Java (JDK 25 โดย compile ให้รองรับ Java 17 ขึ้นไป)
GUI	Java Swing (JFrame, JPanel, JDialog, JLabel, JButton, JOptionPane) และ Java2D สำหรับวาดกราฟิก
Event handling	java.awt.event (KeyAdapter, KeyEvent, ActionListener) และ javax.swing.Timer
การโหลดรูปภาพ	javax.imageio.ImageIO
เครื่องมือเขียนโค้ด	Visual Studio Code และส่วนขยาย Java
การจัดการเวอร์ชัน	Git และ GitHub
แผนภาพ UML	PlantUML
กราฟิก	รูปนักสำรวจ ประตูภารกิจ และประตูชัยจาก Storyboard ใน Proposal ส่วนพื้น กำแพง และผู้พิทักษ์สร้างเป็น pixel art ด้วย Python และไลบรารี Pillow
การสร้างไฟล์เกม	สคริปต์ build.bat (javac และ jar) ได้ไฟล์ NumberMaze.jar ไฟล์เดียวที่รวมโค้ดและรูปภาพไว้แล้ว

2.4 อธิบายส่วนของโปรแกรมตามแนวคิดเชิงวัตถุ

2.4.1 Constructor

```
public abstract class GameObject {
    protected int x, y;


    public GameObject(int x, int y) {
        this.x = x;
        this.y = y;
    }
}


public class NumberItem extends Item {
    private final int value;


    public NumberItem(int x, int y, int value) {
        super(x, y);            // เรียก Constructor ของคลาสแม่
        this.value = value;
    }
}
```

Constructor ของคลาส GameObject รับตำแหน่ง x, y มาตั้งค่า Attribute เมื่อคลาส NumberItem สืบทอดต่อมา (ผ่านคลาส Item) จึงไม่ต้องกำหนดตำแหน่งเอง แต่เรียก super(x, y) ให้ Constructor ของคลาสแม่จัดการ แล้วกำหนดเฉพาะค่า value ที่เป็นของตัวเอง นอกจากนี้ Constructor ของคลาส Level ยังทำหน้าที่อ่านแผนที่และสร้าง Object ทั้งหมดในด่าน และ Constructor ของ PuzzleDialog สร้าง Component ทั้งหมดของหน้าต่างแก้สมการ

2.4.2 Encapsulation

```
public class Inventory {
    private final List<Item> items = new ArrayList<>();


    public void add(Item item)    { items.add(item); }
    public void remove(Item item) { items.remove(item); }


    public List<Item> getItems() {
        return Collections.unmodifiableList(items);   // อ่านได้อย่างเดียว
    }
}


public abstract class Door extends GameObject implements Interactable {
    private boolean locked = true;


    public boolean isLocked() { return locked; }
    protected void unlock()   { locked = false; }
}
```

Attribute ของคลาสเป็นแบบ private หรือ protected
เพื่อป้องกันไม่ให้คลาสอื่นแก้ไขข้อมูลโดยตรง เช่น คลาส Inventory เก็บไอเทมไว้ใน List ที่เป็น private และเมธอด getItems() คืนค่าเป็น List แบบอ่านอย่างเดียว

คลาสอื่นจึงเพิ่มหรือลบไอเทมได้ผ่านเมธอด add() และ remove() เท่านั้น ส่วนคลาส Door กำหนดให้ unlock() เป็น protected ทำให้มีเพียงคลาสลูกอย่าง PuzzleDoor ที่ปลดล็อกประตูได้เมื่อผู้เล่นแก้สมการถูกต้อง

2.4.3 Composition

```
public class Player extends GameObject {
    private final Inventory inventory = new Inventory();
}


public class Level {
    private final List<Item> items = new ArrayList<>();
    private final List<Door> doors = new ArrayList<>();
    private final List<Obstacle> obstacles = new ArrayList<>();
}


public class GamePanel extends JPanel {
    private Level level;
    private Player player;
}
```

คลาส Player มี Attribute เป็น Object ของคลาส Inventory ไว้เก็บไอเทมที่ผู้เล่นเก็บได้ คลาส Level มี List ของ Item, Door และ Obstacle ที่อยู่ในด่าน และคลาส GamePanel มี Object ของ Level และ Player เมื่อเริ่มด่านใหม่ GamePanel จะสร้าง Level และ Player ขึ้นใหม่ ส่วนประกอบเดิมทั้งหมดจึงหายไปพร้อมกัน ทำให้ไอเทม ประตู และกระเป๋ากลับสู่สถานะเริ่มต้น

2.4.4 Polymorphism

```
// GamePanel.tryMove() — เมื่อผู้เล่นเดินชนประตู
Door door = level.findDoorAt(newX, newY);
if (door != null && door.isLocked()) {
    door.interact(player, this);
}


// GamePanel.tick() — ทุก 0.2 วินาที
for (Obstacle obstacle : level.getObstacles()) {
    obstacle.update(level);
}


// GamePanel.paintComponent() — วาดหน้าจอ
for (Item item : level.getItems())             item.draw(g, TILE);
for (Door door : level.getDoors())             door.draw(g, TILE);
for (Obstacle obstacle : level.getObstacles()) obstacle.draw(g, TILE);
```

คลาส GamePanel อ้างถึงวัตถุผ่านตัวแปรชนิดคลาสแม่ เช่น Door, Obstacle และ Item โดยไม่ต้องรู้ว่าแต่ละ Object เป็นคลาสลูกชนิดใด เมื่อเรียก door.interact() ถ้า Object นั้นเป็น PuzzleDoor จะเปิดหน้าต่างแก้สมการ แต่ถ้าเป็น ExitDoor จะผ่านด่าน เช่นเดียวกับ obstacle.update() ที่ทำให้หนามโผล่–หด และผู้พิทักษ์เดิน
ข้อดีคือหากต้องการเพิ่มประตูหรืออุปสรรคชนิดใหม่ ก็เพียงสร้างคลาสลูกใหม่

โดยไม่ต้องแก้ไขโค้ดในคลาส GamePanel นอกจากนี้ยังมีการ override เมธอด paintComponent() ของ JPanel เพื่อวาดหน้าจอเกมเอง

2.4.5 Abstract

```
public abstract class Obstacle extends GameObject {
    public abstract void update(Level level);
    public abstract boolean isDangerous();
    public abstract String getHitMessage();
}

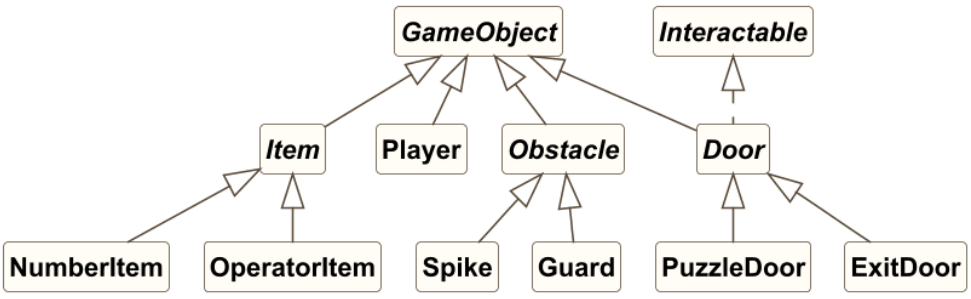

public abstract class Item extends GameObject {
    public abstract String getSymbol();
    public abstract Color getColor();


    public void drawAt(Graphics g, int px, int py, int size) {
        g.setColor(getColor());       // สีของแต่ละคลาสลูก
        ...
        g.drawString(getSymbol(), textX, textY);
    }
}


public interface Interactable {
    void interact(Player player, GamePanel game);
}
```

โปรแกรมมี Abstract class 4 คลาส คือ GameObject, Item, Door และ Obstacle ซึ่งสร้าง Object โดยตรงไม่ได้ เพราะเป็นเพียงแนวคิดรวมของวัตถุ เช่น "อุปสรรค" ที่ยังไม่รู้ว่าเป็นหนามหรือผู้พิทักษ์ จึงกำหนดเพียงว่าคลาสลูกต้องมีเมธอดอะไรบ้าง (Abstract method) คลาส Item ยังมีเมธอด drawAt() ที่เขียนไว้ครั้งเดียวในคลาสแม่ แต่เรียกใช้ getColor() และ getSymbol() ที่คลาสลูกเป็นผู้กำหนด ทำให้ไม่ต้องเขียนโค้ดวาดซ้ำในทุกคลาส ส่วน Interactable เป็น interface ที่กำหนดเพียงเมธอด interact() ให้คลาสที่ implement นำไปเขียนเอง

2.4.6 Inheritance



รูปที่ 2.5 แผนภาพการสืบทอดคลาสของวัตถุในเกม

```
public class Player extends GameObject { ... }
public class NumberItem extends Item { ... }
public class PuzzleDoor extends Door { ... }
public class Spike extends Obstacle { ... }


public class GamePanel extends JPanel { ... }       // สืบทอดจาก Swing
public class PuzzleDialog extends JDialog { ... }   // สืบทอดจาก Swing
```

วัตถุทุกชนิดในเกมสืบทอดจากคลาส GameObject จึงได้ Attribute x, y และเมธอด getX(), getY() ไปใช้ได้ทันทีโดยไม่ต้องเขียนซ้ำ การสืบทอดมี 2 ชั้น เช่น GameObject ⟶ Item ⟶ NumberItem และ GameObject ⟶ Door ⟶ PuzzleDoor นอกจากนี้คลาส GamePanel สืบทอดจาก JPanel และ PuzzleDialog สืบทอดจาก JDialog ของ Swing ทำให้ได้ความสามารถของหน้าต่างและพื้นที่วาดภาพมาใช้ แล้วเพิ่มเติมเฉพาะส่วนที่เกมต้องการ

2.5 หน้าจอ GUI

เกมมีหน้าจอหลัก 2 ส่วน คือ หน้าต่างเกม (JFrame ที่มี GamePanel) และหน้าต่างแก้สมการ (PuzzleDialog) นอกจากนี้ยังใช้กล่องข้อความ JOptionPane แจ้งเมื่อตอบถูกและเมื่อผ่านด่าน

2.5.1 หน้าต่างเกม (GamePanel)



รูปที่ 2.6 หน้าต่างเกมระหว่างเล่น พร้อมหมายเลขส่วนประกอบ

ตารางที่ 2.6 ส่วนประกอบของหน้าต่างเกม

ที่	ส่วนประกอบ	คำอธิบาย
1	นักสำรวจ (Player)	ตัวละครที่ผู้เล่นควบคุม วาดด้วยรูปจาก Storyboard
2	ไอเทมตัวเลข (NumberItem)	กล่องสีเหลือง เดินทับเพื่อเก็บ
3	ไอเทมเครื่องหมาย (OperatorItem)	กล่องสีฟ้า เดินทับเพื่อเก็บ
4	ประตูภารกิจ (PuzzleDoor)	ป้ายสีดำด้านล่างแสดงเป้าหมาย เมื่อเปิดแล้วจะเหลือภาพจาง ๆ
5	ประตูชัย (ExitDoor)	ทางออกของด่าน
6	กำแพง	พื้นที่ที่เดินผ่านไม่ได้
7	ชื่อด่าน	ชื่อด่านปัจจุบันและลำดับด่าน เช่น (1/3)
8	ข้อความแจ้งเตือน	เช่น "เก็บ 4 แล้ว" หรือ "โดนหนาม! กลับไปจุดเริ่มต้น"
9	คำแนะนำปุ่มลัด	บอกว่ากด R เพื่อเริ่มด่านใหม่
10	กระเป๋า (Inventory)	แสดงไอเทมทั้งหมดที่ผู้เล่นเก็บได้

หน้าต่างเกมไม่ได้ใช้ Component ของ Swing วางเรียงกัน แต่ใช้ JPanel เพียงตัวเดียว (GamePanel) แล้ว override เมธอด paintComponent() เพื่อวาดทุกอย่างด้วย Java2D ตามลำดับชั้น คือ พื้นและกำแพง ⟶ ไอเทม ⟶ ประตู ⟶ อุปสรรค ⟶ ผู้เล่น ⟶ แถบข้อมูลด้านล่าง (HUD) และหน้าจอชนะ ทุกครั้งที่สถานะเกมเปลี่ยน จะเรียก repaint() เพื่อวาดหน้าจอใหม่ ขนาดของหน้าต่างปรับตามขนาดแผนที่ของแต่ละด่านด้วย pack()

2.5.2 หน้าต่างแก้สมการ (PuzzleDialog)



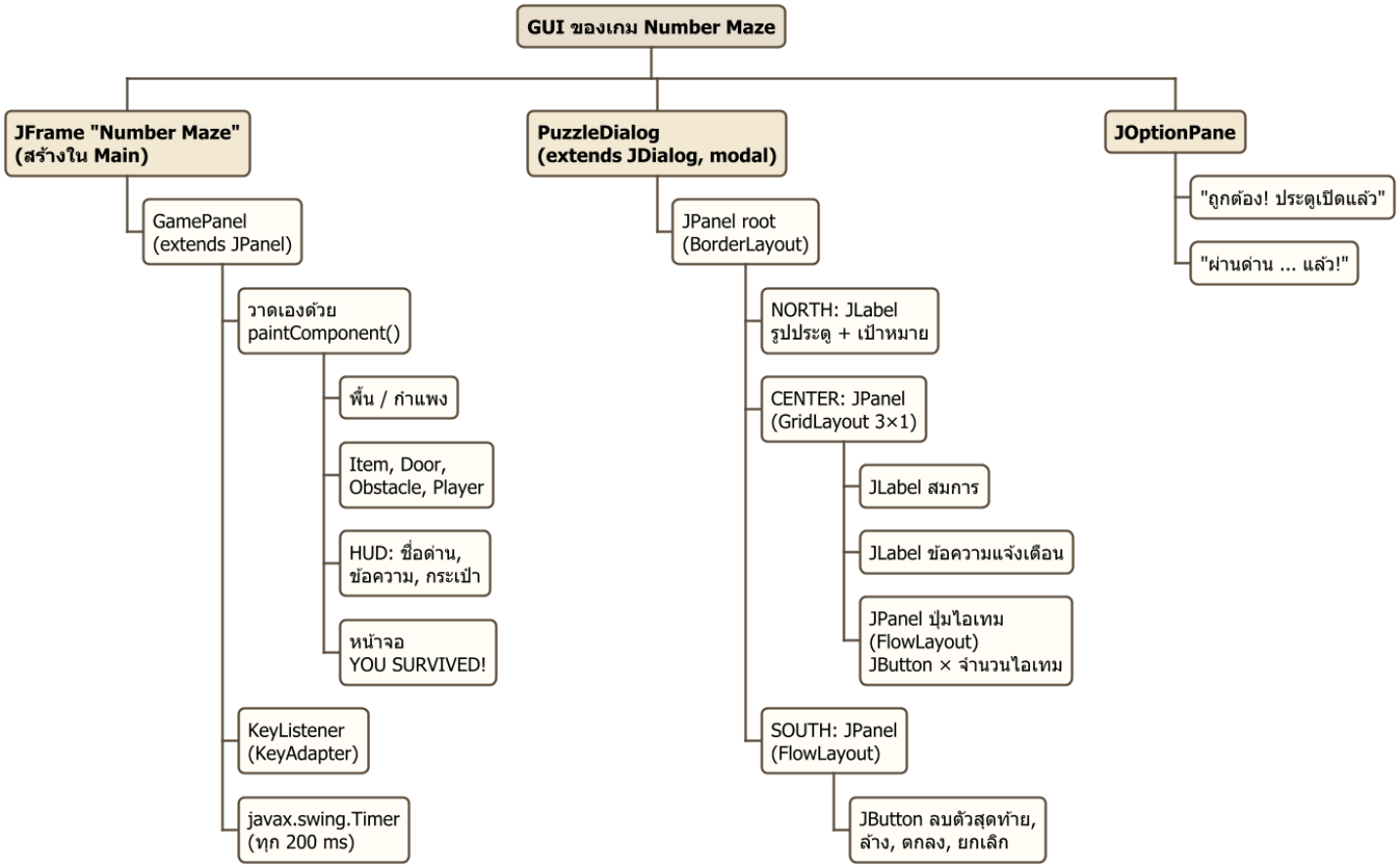
รูปที่ 2.7 หน้าต่างแก้สมการ พร้อมหมายเลขส่วนประกอบ (ตัวอย่างตอนตอบผิด)

ตารางที่ 2.7 Component ของหน้าต่างแก้สมการ

ที่	Component	หน้าที่
1	JLabel (มี ImageIcon)	แสดงรูปประตูภารกิจและเป้าหมาย อยู่ตำแหน่ง NORTH ของ BorderLayout
2	JLabel	แสดงสมการที่ผู้เล่นเรียงอยู่ ถ้ายังไม่เลือกจะแสดง "?"
3	JLabel (ตัวอักษรสีแดง)	แสดงผลการตรวจ เช่น "ได้ 10 ยังไม่ตรงกับเป้าหมาย 16" หรือข้อผิดพลาดของรูปแบบสมการ
4	JPanel (FlowLayout) + JButton	ปุ่มไอเทมตามจำนวนในกระเป๋า ไอเทมที่เลือกไปแล้วจะกดซ้ำไม่ได้ (setEnabled(false))
5	JPanel (FlowLayout) + JButton 4 ปุ่ม	ลบตัวสุดท้าย / ล้าง / ตกลง / ยกเลิก อยู่ตำแหน่ง SOUTH

หน้าต่างนี้เป็น JDialog แบบ Modal (ModalityType.APPLICATION_MODAL) คือระหว่างที่หน้าต่างเปิดอยู่ ผู้เล่นจะกดอะไรในหน้าต่างเกมไม่ได้ และโค้ดที่เรียก setVisible(true) จะหยุดรอจนกว่าหน้าต่างจะปิด ส่วนประกอบกลางจัดด้วย GridLayout 3 แถว 1 คอลัมน์ และใช้ JOptionPane แสดงข้อความ "ถูกต้อง! ประตูเปิดแล้ว" เมื่อตอบถูก

2.5.3 โครงสร้างของ GUI



รูปที่ 2.8 โครงสร้าง Component ของ GUI ทั้งหมดในเกม

จากรูปโครงสร้าง GUI หน้าต่างหลักเป็น JFrame ที่มี GamePanel เป็น Component เดียว ซึ่งรับผิดชอบทั้งการวาดหน้าจอ การรับปุ่มกด (KeyListener) และนาฬิกาของเกม (Timer) ส่วน PuzzleDialog จัดวาง Component ซ้อนกันเป็นลำดับชั้นด้วย Layout Manager 3 แบบ คือ BorderLayout แบ่งหน้าต่างเป็นบน กลาง ล่าง, GridLayout เรียงส่วนกลางเป็น 3 แถวเท่ากัน และ FlowLayout เรียงปุ่มจากซ้ายไปขวา

2.5.4 หน้าจออื่น ๆ



รูปที่ 2.9 ด่านที่ 2 ทางหนาม ต้องรอจังหวะที่หนามหดจึงเดินผ่านได้



รูปที่ 2.10 ด่านที่ 3 ผู้พิทักษ์ แผนที่ขนาดใหญ่ มีผู้พิทักษ์ลาดตระเวนและไอเทมหลอก



รูปที่ 2.11 หน้าจอ YOU SURVIVED! เมื่อผ่านประตูชัยของด่านสุดท้าย

2.6 Event handling ในหน้าจอ

เกมใช้ Event 3 ประเภท คือ KeyEvent จากคีย์บอร์ด, ActionEvent จากการคลิกปุ่ม และ ActionEvent จาก javax.swing.Timer สรุปได้ดังตารางต่อไปนี้

ตารางที่ 2.8 Event handling ทั้งหมดในโปรแกรม

แหล่งของ Event	Event / Listener	เมธอดที่ทำงาน	ผลลัพธ์
GamePanel (คีย์บอร์ด)	KeyEvent / KeyAdapter	keyPressed() ⟶ tryMove()	W A S D / ลูกศร: เดิน เก็บไอเทม หรือชนประตู
GamePanel (คีย์บอร์ด)	KeyEvent / KeyAdapter	keyPressed() ⟶ loadLevel()	R: เริ่มด่านใหม่, Enter: เล่นใหม่หลังจบเกม
javax.swing.Timer	ActionEvent (ทุก 200 ms)	tick()	อุปสรรคขยับ ตรวจการชน แล้ววาดจอใหม่
ปุ่มไอเทม (JButton)	ActionEvent / lambda	equation.add() ⟶ refresh()	เพิ่มไอเทมต่อท้ายสมการ
ปุ่ม "ลบตัวสุดท้าย"	ActionEvent / lambda	equation.remove() ⟶ refresh()	ลบไอเทมตัวสุดท้ายออกจากสมการ
ปุ่ม "ล้าง"	ActionEvent / lambda	equation.clear() ⟶ refresh()	ล้างสมการทั้งหมด
ปุ่ม "ตกลง"	ActionEvent / lambda	submit()	ตรวจคำตอบ ถ้าถูกลบไอเทมที่ใช้และปิดหน้าต่าง
ปุ่ม "ยกเลิก"	ActionEvent / lambda	dispose()	ปิดหน้าต่าง ไอเทมยังอยู่ครบ

2.6.1 การรับปุ่มกดจากคีย์บอร์ด (KeyListener)

```
setFocusable(true);              // ต้องเปิด JPanel จึงจะรับปุ่มกดได้
addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (gameWon) {
            if (e.getKeyCode() == KeyEvent.VK_ENTER) {
                gameWon = false;
                loadLevel(0);
            }
            return;
        }
        switch (e.getKeyCode()) {
            case KeyEvent.VK_W, KeyEvent.VK_UP    -> tryMove(0, -1);
            case KeyEvent.VK_S, KeyEvent.VK_DOWN  -> tryMove(0, 1);
            case KeyEvent.VK_A, KeyEvent.VK_LEFT  -> tryMove(-1, 0);
            case KeyEvent.VK_D, KeyEvent.VK_RIGHT -> tryMove(1, 0);
            case KeyEvent.VK_R -> {
                loadLevel(levelIndex);
                message = "เริ่มด่านใหม่แล้ว";
            }
        }
    }
});
```

GamePanel ใช้ Anonymous class ที่สืบทอดจาก KeyAdapter และ override เฉพาะ keyPressed() (ไม่ต้อง implement keyReleased() และ keyTyped() เหมือนการใช้ KeyListener โดยตรง) จากนั้นแยกการทำงานตามรหัสปุ่มด้วย switch ทั้งนี้ JPanel ต้องเรียก setFocusable(true) จึงจะได้รับ KeyEvent และหลังจากปิดหน้าต่างแก้สมการ ต้องเรียก requestFocusInWindow() เพื่อดึงโฟกัสของคีย์บอร์ดกลับมาที่เกม

2.6.2 นาฬิกาของเกม (javax.swing.Timer)

```
timer = new Timer(200, e -> tick());     // ส่ง ActionEvent ทุก 200 ms
timer.start();


private void tick() {
    for (Obstacle obstacle : level.getObstacles()) {
        obstacle.update(level);
    }
    checkObstacles();
    repaint();
}
```

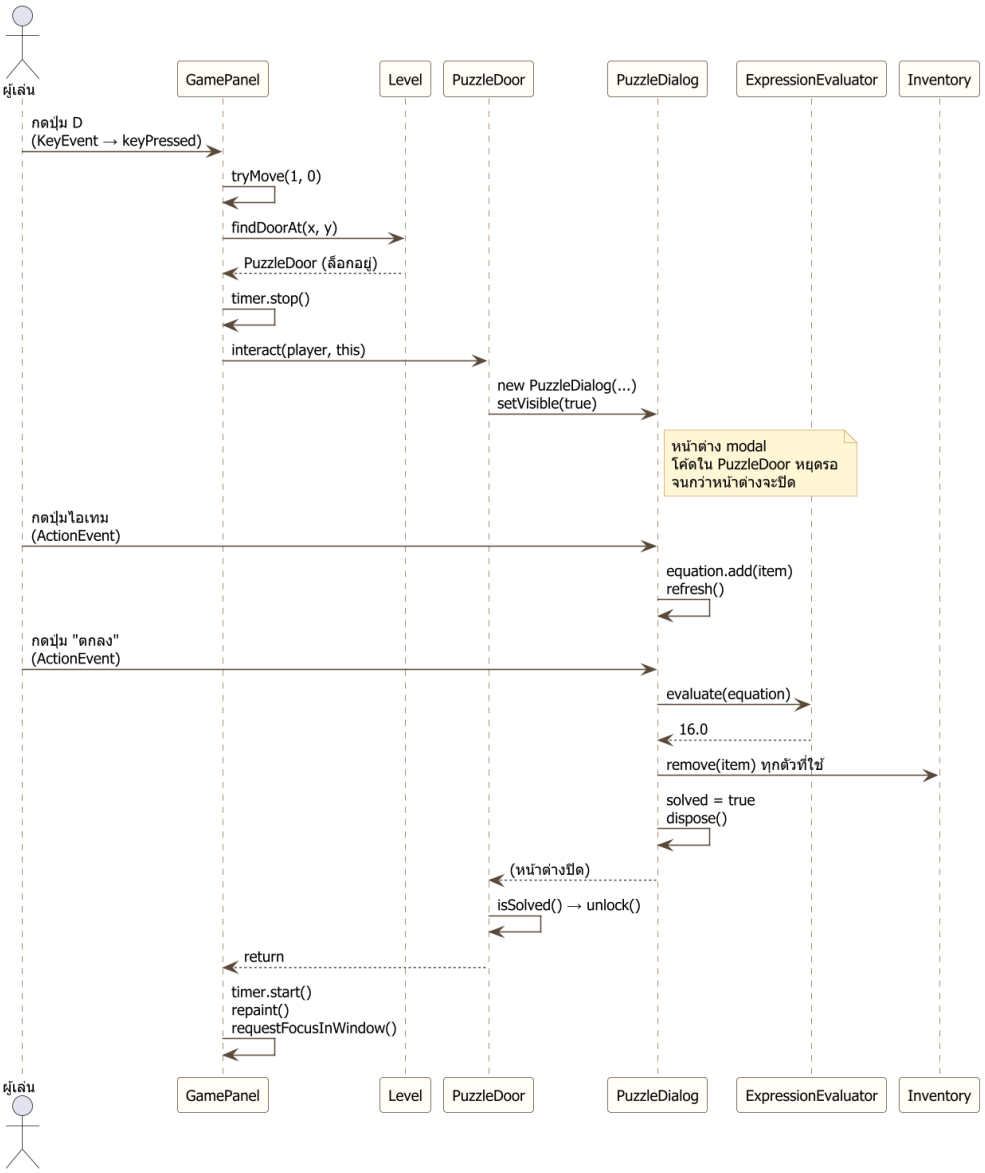
Timer ของ Swing ส่ง ActionEvent ทุก 200 มิลลิวินาทีบน Event Dispatch Thread เดียวกับ Event อื่นของ Swing จึงแก้ไขข้อมูลของเกมได้อย่างปลอดภัยโดยไม่ต้องจัดการ Thread เอง ในแต่ละ tick อุปสรรคทุกตัวจะเรียก update() ของตัวเอง
แล้วตรวจว่าผู้เล่นโดนอุปสรรคหรือไม่ ระหว่างที่หน้าต่างแก้สมการเปิด จะหยุด Timer ด้วย timer.stop() เพื่อไม่ให้ผู้พิทักษ์เดินมาชนผู้เล่น

2.6.3 การคลิกปุ่มในหน้าต่างแก้สมการ (ActionListener)

```
for (Item item : inventory.getItems()) {
    JButton b = new JButton(item.getSymbol());
    b.setEnabled(!equation.contains(item));   // เลือกแล้วกดซ้ำไม่ได้
    b.addActionListener(e -> {
        equation.add(item);
        messageLabel.setText(" ");
        refresh();
    });
    itemPanel.add(b);
}


undoButton.addActionListener(e -> { ... equation.remove(equation.size() - 1);
refresh(); });
clearButton.addActionListener(e -> { equation.clear(); refresh(); });
submitButton.addActionListener(e -> submit());
cancelButton.addActionListener(e -> dispose());
```

ปุ่มทุกปุ่มใช้ Lambda expression เป็น ActionListener ซึ่งสั้นกว่าการสร้างคลาสที่ implement ActionListener ปุ่มไอเทมถูกสร้างใหม่ทุกครั้งที่เรียก refresh() ตามไอเทมในกระเป๋า และปุ่มของไอเทมที่อยู่ในสมการแล้วจะถูก setEnabled(false)
ลำดับการทำงานทั้งหมดตั้งแต่กดปุ่มเดินจนประตูเปิด แสดงในรูปที่ 2.12



รูปที่ 2.12 Sequence Diagram ของ Event เมื่อผู้เล่นเดินชนประตูภารกิจแล้วแก้สมการถูก

2.7 อัลกอริทึมที่สำคัญในโปรแกรม

2.7.1 การคำนวณสมการตามลำดับความสำคัญทางคณิตศาสตร์

เกมกำหนดให้คำนวณตามหลักคณิตศาสตร์จริง คือต้องคำนวณคูณและหารก่อนบวกและลบ เช่น 3 + 2 × 4 ต้องได้ 11 ไม่ใช่ 20 คลาส ExpressionEvaluator จึงคำนวณเป็น 2 รอบ ดังนี้

1. รอบที่ 1: ไล่สมการจากซ้ายไปขวา ถ้าเจอ × หรือ ÷ ให้คำนวณกับตัวเลขตัวล่าสุดทันที ถ้าเจอ + หรือ − ให้เก็บเครื่องหมายและตัวเลขไว้ก่อน
2. รอบที่ 2: นำตัวเลขที่เหลือมาบวกหรือลบกันจากซ้ายไปขวา

ตารางที่ 2.9 ตัวอย่างการคำนวณ 3 + 2 × 4

ขั้นตอน	สิ่งที่ทำ	numbers	operators
เริ่มต้น	เก็บตัวเลขตัวแรก	[3]	[ ]
รอบที่ 1	เจอ + 2 ⟶ ยังไม่คำนวณ เก็บไว้ก่อน	[3, 2]	[+]
รอบที่ 1	เจอ * 4 ⟶ คำนวณทันที 2 × 4 = 8	[3, 8]	[+]
รอบที่ 2	บวก/ลบจากซ้ายไปขวา 3 + 8	ผลลัพธ์ = 11	

```
// รอบที่ 1: * และ /
for (int i = 1; i < tokens.size(); i += 2) {
    char op = ((OperatorItem) tokens.get(i)).getOperator();
    double next = valueOf(tokens.get(i + 1));
    int last = numbers.size() - 1;


    if (op == '*') {
        numbers.set(last, numbers.get(last) * next);
    } else if (op == '/') {
        if (next == 0) throw new ArithmeticException("หารด้วยศูนย์ไม่ได้");
        numbers.set(last, numbers.get(last) / next);
    } else {
        operators.add(op);                      // + หรือ - เก็บไว้ทำรอบที่ 2
        numbers.add(next);
    }
}


// รอบที่ 2: + และ -
double result = numbers.get(0);
for (int i = 0; i < operators.size(); i++) {
    if (operators.get(i) == '+') result += numbers.get(i + 1);
    else                         result -= numbers.get(i + 1);
}
```

2.7.2 การตรวจรูปแบบสมการ

ก่อนคำนวณ เมธอด validate() ตรวจว่าสมการถูกรูปแบบ คือตำแหน่งคู่ (0, 2, 4, ...) ต้องเป็นตัวเลข และตำแหน่งคี่ (1, 3, 5, ...) ต้องเป็นเครื่องหมาย สมการต้องขึ้นต้นและจบด้วยตัวเลข และต้องมีเครื่องหมายอย่างน้อย 1 ตัว หากไม่ถูกต้องจะ throw IllegalArgumentException พร้อมข้อความ แล้ว PuzzleDialog จะ catch ไปแสดงให้ผู้เล่นเห็น
ไอเทมจะถูกลบออกจากกระเป๋าเฉพาะเมื่อคำตอบถูกต้องเท่านั้น

2.7.3 การเคลื่อนที่และการตรวจการชน

ทุกครั้งที่ผู้เล่นกดปุ่มเดิน เมธอด tryMove(dx, dy)
จะคำนวณช่องปลายทางแล้วตรวจตามลำดับ ดังนี้

1. ถ้าช่องปลายทางเป็นกำแพง ⟶ ไม่ขยับ
2. ถ้ามีประตูที่ยังล็อกอยู่ ⟶ หยุด Timer เรียก door.interact() แล้วเริ่ม Timer ใหม่ (ผู้เล่นยังไม่เดินเข้าไป)

3. ถ้าไม่ใช่ทั้งสองกรณี ⟶ ย้ายผู้เล่น ถ้าช่องนั้นมีไอเทมให้ย้ายจากแผนที่เข้ากระเป๋า

4. ตรวจว่าผู้เล่นยืนอยู่ช่องเดียวกับอุปสรรคที่กำลังอันตรายหรือไม่ ถ้าใช่ส่งกลับจุดเริ่มต้น

สำหรับอุปสรรค Spike ใช้ตัวนับ tick วนรอบ 11 จังหวะ โดยอันตรายเมื่อ tick < 5 (โผล่ 1 วินาที หด 1.2 วินาที) ส่วน Guard ขยับทุก 2 tick
ถ้าช่องข้างหน้าเป็นกำแพงหรือประตูจะกลับทิศทาง (direction = -direction)

2.7.4 การสร้างด่านจากแผนที่แบบข้อความ

แผนที่ของแต่ละด่านเขียนเป็นอาร์เรย์ของข้อความ (String[])
โดยแต่ละตัวอักษรแทนสิ่งที่อยู่ในช่องนั้น Constructor ของคลาส Level
จะอ่านทีละตัวอักษรแล้วสร้าง Object ที่ตรงกัน
ทำให้การออกแบบหรือแก้ไขด่านทำได้ง่ายโดยไม่ต้องแก้ไขโค้ดส่วนอื่น

ตารางที่ 2.10 ตัวอักษรในแผนที่และ Object ที่สร้าง

ตัวอักษร	สิ่งที่สร้าง
`#` `.`	กำแพง / ทางเดิน
`P`	จุดเริ่มต้นของผู้เล่น
`0–9`	NumberItem (ไอเทมตัวเลข)
`+ - * /`	OperatorItem (ไอเทมเครื่องหมาย)
`a–z`	PuzzleDoor (ประตูภารกิจ) โดยเป้าหมายกำหนดใน Map เช่น Map.of('a', 16)
`E`	ExitDoor (ประตูชัย)
`^`	Spike (กับดักหนาม)
`G`	Guard (ผู้พิทักษ์)

```
"####################",
"#P..#.^^.2#..+..#.^8#",
"#.#.#.##..#.###.#.###",
"#.#...#-..a...G.....#",       // a = ประตูภารกิจ, G = ผู้พิทักษ์
...
```

2.7.5 การตรวจสอบว่าทุกด่านเล่นจบได้

เนื่องจากไอเทมที่ใช้แล้วจะหายไป ด่านที่ออกแบบไม่ดีอาจทำให้ผู้เล่นไม่มีไอเทมพอจะผ่าน ผู้จัดทำจึงเขียนโปรแกรมตรวจสอบด่านแยกต่างหาก โดยใช้ Breadth-First Search (BFS) หาพื้นที่ที่เดินถึงได้จากจุดเริ่มต้นและรวบรวมไอเทมในพื้นที่นั้น จากนั้นทดลองเรียงไอเทมทุกแบบ (Brute force) เพื่อหาคำตอบของประตูที่เจอ เมื่อเปิดประตูได้จึงทำ BFS ต่อในพื้นที่ใหม่ วนไปจนถึงประตูชัย ผลการตรวจยืนยันว่าทั้ง 3 ด่านเล่นจบได้ และประตูทุกบานในด่านที่ 3 ต้องใช้ไอเทมอย่างน้อย 5 ชิ้น

บทที่ 3 สรุป

ผู้จัดทำได้พัฒนาเกมเขาวงกตตัวเลขด้วยภาษา Java ครบตาม Proposal และ Storyboard ที่ออกแบบไว้ ประกอบด้วย 3 ด่านที่มีความยากเพิ่มขึ้นตามลำดับ ระบบเก็บไอเทม ประตูภารกิจที่ต้องแก้สมการ อุปสรรค 2 ชนิด และหน้าจอ "YOU SURVIVED!" เมื่อจบเกม โดยนำแนวคิด Constructor, Encapsulation, Composition, Polymorphism, Abstract และ Inheritance มาใช้ในการออกแบบคลาส และเผยแพร่ Source code พร้อมไฟล์ jar บน GitHub

3.1 ปัญหาที่พบระหว่างการพัฒนา

1. ข้อความภาษาไทยในกล่องข้อความ (JOptionPane) แสดงเป็นสี่เหลี่ยม
   เนื่องจากฟอนต์เริ่มต้นของ Swing ไม่รองรับภาษาไทย แก้ไขโดยเปลี่ยนฟอนต์เริ่มต้นของทุก Component เป็น Tahoma ผ่าน UIManager ในเมธอด useThaiFont()
2. เมื่อไอเทมถูกใช้หมดไปหลังตอบถูก ผู้เล่นที่ใช้ไอเทมผิดประตูอาจไม่มีไอเทมพอจะผ่านด่าน จึงเพิ่มปุ่ม R สำหรับเริ่มด่านใหม่
3. การออกแบบด่านให้เล่นจบได้จริงและยากพอเหมาะทำได้ยาก ด่านที่ 3 รุ่นแรกมีคำตอบง่ายเกินไป (เช่น 9 – 4) จึงขยายแผนที่ เพิ่มไอเทมหลอก ปรับโจทย์ และเขียนโปรแกรมตรวจสอบด่านด้วย BFS เพื่อยืนยัน
4. แต่ละด่านมีขนาดแผนที่ไม่เท่ากัน ทำให้หน้าต่างเกมไม่พอดีกับแผนที่ แก้ไขโดยเรียก pack() ของหน้าต่างทุกครั้งที่เปลี่ยนด่าน
5. ผู้พิทักษ์ยังคงเดินในขณะที่ผู้เล่นกำลังแก้สมการ แก้ไขโดยหยุด Timer ระหว่างที่หน้าต่างแก้สมการเปิดอยู่ และหลังปิดหน้าต่างต้องเรียก requestFocusInWindow() เพื่อให้คีย์บอร์ดกลับมาควบคุมเกมได้

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

1. **ตัวเลขเป็นทรัพยากรที่ใช้แล้วหมดไป** ต่างจากเกมคิดเลขทั่วไปที่ให้ตัวเลขมาครบ ผู้เล่นต้องเดินเก็บเองและวางแผนว่าจะใช้ชิ้นไหนกับประตูไหน เพราะบางชิ้นต้องเก็บไว้ใช้ในห้องถัดไป และมีไอเทมหลอกที่ไม่ต้องใช้
2. **คำนวณตามลำดับความสำคัญทางคณิตศาสตร์จริง** ผู้เล่นต้องคิดว่า 7 × 3 − 5 กับ 7 + 3 × 5 ได้ผลต่างกัน ช่วยฝึกเรื่องที่มักคำนวณผิด
3. **ผสมการคิดกับจังหวะเวลา** มีทั้งส่วนที่ต้องคิดเลข และส่วนที่ต้องรอจังหวะหลบหนามหรือผู้พิทักษ์
4. **ด่านทุกด่านผ่านการตรวจด้วยโปรแกรม** ยืนยันได้ว่าเล่นจบได้จริงและไม่มีทางลัดแบบง่าย

5. **ออกแบบด่านด้วยข้อความ** เพิ่มหรือแก้ด่านได้โดยพิมพ์แผนที่ใหม่ โดยไม่ต้องแก้โค้ดส่วนอื่น และกราฟิกตัวละครมาจาก Storyboard ของโครงงานเอง

3.3 คำแนะนำสำหรับผู้สอน

อยากให้อาจารย์กำหนดเวลาในการส่งงานชัดเจนมากขึ้น